Типы данных с плавающей точкой: float, double и long double

На этом уроке мы рассмотрим типы данных с плавающей точкой в языке C++, их точность и диапазон. Выясним, что такое всего ненадежная запись и как она используется, а также рассмотрим ошибки округления и задим определения для `nan` и `inf`.

Оглавление:

1. **Типы данных с плавающей точкой**
2. **Экспоненциальная запись**
3. **Конвертация чисел в экспоненциальную запись**
4. **Точность и диапазон типов с плавающей точкой**
5. **Ошибки округления**
6. **`nan` и `inf`**
7. **Заключение**
8. **Тест**

Типы данных с плавающей точкой

Целочисленные типы данных отлично подходят для работы с целыми числами, но есть ведь еще и дробные числа. И тут нам на помощь приходят **типы данных с плавающей точкой** (или **«типы данных с плавающей запятой»**, англ. **floating points**). Переменные такого типа может хранить любые действительные дробные значения, например: 4320.0, 3.33 или 0.0126. Почему точка называющая? Дело в том, что точка/запятая перемещается («плавает») между цифрами, разделяя целую и дробную части значения.

Есть три типа данных с плавающей точкой: **float**, **double** и **long double**. Язык C++ определяет только их минимальный размер (как и с целочисленными типами). Типы данных с плавающей точкой всегда являются знаковыми (т.е. могут хранить как положительные, так и отрицательные числа).

	Тип	Минимальный размер	Типичный размер
Тип данных с плавающей точкой	float	4 байта	4 байта
	double	8 байт	8 байт
	long double	8 байт	8, 12 или 16 байт

Объявление переменных разных типов данных с плавающей точкой:

```
float f1(0.0);
double d1(0.0);
long double ld1(0.0);
```

Если нужно использовать целое число с переменной типа с плавающей точкой, то тогда после этого числа нужно поставить разделительную точку и нуль. Это позволяет различить переменные целочисленных типов от переменных типов с плавающей запятой:

```
int i(1);
double d1(1.0); // 1.0 - это тип double с плавающей точкой (по умолчанию double)
float f1(1.0f); // 1.0f - это тип float с плавающей точкой (1.0f => float)
```

Обратите внимание, литералы типа с плавающей точкой по умолчанию относятся к типу double. `f` в конце числа означает тип float.

Экспоненциальная запись

Экспоненциальная запись очень полезна для написания длинных чисел в краткой форме. Числа в экспоненциальной записи имеют следующий вид: **мантисса** x $10^{экспонента}$. Например, рассмотрим **выражение** 1.2 x 10^5. Значение 1.2 — это **мантисса** (или **«значащая часть мантиссы»**), а 5 — это **экспонента** (или **«порядок числа»**). Результатом этого выражения является значение 120000.

Обычно, в экспоненциальной записи, в целой части находится только одна цифра, все остальные лишутся после разделительной точки (в дробной части).

Рассмотрим массу Земли. В десятичной системе счисления она представлена как 5972000000000000000000000 кг. Согласитесь, очень большое число (даже слишком большое, чтобы поместиться в целочисленную переменную размером 8 байт). Это число даже трудно читать (где 19 или 20 нулей?). Но использая экспоненциальную запись, массу Земли можно представить, как 5.972E + 10^{24} (что гораздо легче воспринимается, согласитесь). Еще одним преимуществом экспоненциальной записи является сравнение двух очень больших или очень маленьких чисел — для этого достаточно просто сравнить их экспоненты.

В языке C++ буква e / E означает, что число 10 нужно возвести в степень, которая следует за этой буквой. Например, 1.2 x 10^5 показывается 1.2e5, значение 5.972E + 10^{24} еще короче выглядит, 5.972E24.

Для чисел меньше единицы экспонент может быть отрицательным. Например, 5x2 эквивалентно 5 x 10^{-2}, что, в свою очередь, означает 5 / 10^2 или 0.05. Масса электрона равна 9.109382e-31 кг.

Не практике экспоненциальная запись может использоваться в операциях приведенным следующим образом:

```
double d(2000.0);
double d2(5e3); // другой способ записать значение 5000

double d3(0.05);
double d4(5e-2); // другой способ записать значение 0.05
```

Конвертация чисел в экспоненциальную запись

Для конвертации чисел в экспоненциальную запись необходимо следовать процедуре, указанной ниже:

- Ваш экспонент начинается с нуля.
- Перемещяем разделительную точку (которая разделяет целую и дробную части) влево, чтобы слева от нее осталась только одна ненулевая цифра:
  - каждое перемещение точки влево увеличивает экспонент на 1;
  - каждое перемещение точки вправо уменьшает экспонент на 1;
- Отнимите все нули перед первой ненулевой цифрой в целой части.
- Отнимите все конечные нули в правой (дробной) части, *только если сособое число является целым (без разделительной точки)*.

Рассмотрим примеры:

```
Исходное число: 42030
Перемещяем разделительную точку на 4 цифры влево: 4.20304
Слева (в целой части) нет нулей: 4.20304
Отбрасываем конечный нуль в дробной части: 4.2034 (4 значащие цифры)

Исходное число: 0.0070500
Перемещяем разделительную точку на 3 цифры вправо: 0007.0500e-3
Отбрасываем нули слева: 7.0500e-3
В язык C++ буква e / E означает, что число 10 нужно возвести в степень, которая следует за этой буквой. Например, 1.2 x
10^5 эквивалентно 1.2e5, значение 5.972E + 10^24 еще короче выглядит, 5.972E24

Для чисел меньше единицы экспонент может быть отрицательным. Например, 5x2 эквивалентно 5 * 10^-2, что, в свою
очередь, означает 5 / 10^2 или 0.05. Масса электрона равна 9.109382e-31 кг.

Не практике экспоненциальная запись может использоваться в операциях приведенным следующим образом:

double d(2000.0);
double d2(5e3); // другой способ записать значение 5000

double d3(0.05);
double d4(5e-2); // другой способ записать значение 0.05
```

Конвертация чисел в экспоненциальную запись

Для конвертации чисел в экспоненциальную запись необходимо следовать процедуре, указанной ниже:

- Ваш экспонент начинается с нуля.
- Перемещяем разделительную точку (которая разделяет целую и дробную части) влево, чтобы слева от нее осталась только одна ненулевая цифра:
  - каждое перемещение точки влево увеличивает экспонент на 1;
  - каждое перемещение точки вправо уменьшает экспонент на 1;
- Отнимите все нули перед первой ненулевой цифрой в целой части.
- Отнимите все конечные нули в правой (дробной) части, *только если сособое число является целым (без разделительной точки)*.

Рассмотрим примеры:

```
Исходное число: 42030
Перемещяем разделительную точку на 4 цифры влево: 4.20304
Слева (в целой части) нет нулей: 4.20304
Отбрасываем конечный нуль в дробной части: 4.2034 (4 значащие цифры)

Исходное число: 0.0070500
Перемещяем разделительную точку на 3 цифры вправо: 0007.0500e-3
Отбрасываем нули слева: 7.0500e-3
Исходное число: 500.410
Перемещяем разделительную точку на 2 цифры влево: 6.00410e2
Слева нет нулей: 6.00410e2
Нуль справа остается: 6.00410e2 (5 значащих цифр)
```

Самое главное, что нужно запомнить — это то, что цифры в мантиссе (часть перед `e`) называются **значащими цифрами**. Количество значащих цифр определяет **точность** самого значения, чем больше цифр в мантиссе, тем точнее значение.

Точность и диапазон типов с плавающей точкой

Рассмотрим дробь 1/3. Десятичное представление этого числа — 0.333333333333... (с тройками до бесконечности). Бесконечное число требует бесконечной памяти для хранения, а у нас в запасе, как правило, 4 или 8 байт. Переменные типа с плавающей запятой могут хранить только определенное количество значащих цифр, остальные — отбрасываются. **Точность определяется количеством значащих цифр**, которые представляет число без потери данных.

Когда мы выводим переменные типа с плавающей точкой, то точность обычно *пол*, по умолчанию, составляет 6. Т.е. на экране мы увидим только 6 значащих цифр, остальные — потеряются. Например:

```
int main()
{
    float f1;
    f = 3.876543;
    cout << f << endl;
    f = 987.654321;
    cout << f << endl;
    f = 9.876543e-85;
    cout << f << endl;
    double d = 3.333333333333333333333333;
    cout << d << endl;
    return 0;
}
```

Результат выполнения программы:

```
3.87654
987.654
987654
9.87654e-85
3.87654e-85
```

Обратите внимание, каждое из вышеприведенных значений имеет только 6 значащих цифр (цифры перед `e`, а не перед точкой).

Также, в некоторых случаях, сол сам может выводить числа в экспоненциальной записи. В зависимости от компьютера, экспонент может быть длиннее нуженго. Например, 9.87654e+06 — это то же самое, что и 9.87654e6 (просто с добавлением нулей в экспоненту). Минимальное количество цифр экспонента определяется компилятором (Visual Studio использует 2, другие компиляторы могут использовать 3).

Также мы можем переопределить точность сол, используя **функцию std::setprecision()**, которая находится в **заголовочном файле** iomanip:

```
#include <iomanip>
#include <iostream> // для std::setprecision()

int main()
{
    cout << std::setprecision(20); // меняем точность с 20 цифр
    float f = 1.333333333333333333333333;
    cout << f << endl;
    double d = 1.333333333333333333333333;
    cout << d << endl;
    return 0;
}
```

Результат выполнения программы:

```
1.33333335388474
1.33333333333333
```

Так как мы увеличили точность до 16, то каждая переменная выводится (16 цифрами. Но, как вы можете видеть, исходные числа имеют больше цифр!

Точность зависит от размера типа данных (в типе float точность меньше, чем в типе double) и от присваиваемого значения:

- **точность float:** от 6 до 9 цифр (в основном 7);
- **точность double:** от 15 до 18 цифр (в основном 16);
- **точность long double:** 15, 18 или 33 цифры (в зависимости от того, сколько байт занимает тип данных на компьютере).

Этот процедин относится не только к дробным числам, но и ко всем значениям, которые имеют слишком большое количество значащих цифр. Например:

```
#include <iomanip>
#include <iostream> // для std::setprecision()

int main()
{
    float f(123456789.0f); // инициализируем f целым 10 значащим цифр
    cout << f << endl; // выводим значение на 6 цифр
    cout << f << endl;
    return 0;
}
```

Результат:

```
123456792
```

Но ведь 123456792 больше чем 123456789, чем так лит? Значение 123456789.0 имеет 10 значащих цифр, но точность float равна 7. Поэтому мы и получили другое число, произошла потеря данных!

Следовательно, нужно быть осторожними, когда вы используете переменные типа с плавающей точкой вместе с очень большимчислами, маленькими числами, которые требуют большой точности, чем их текущий тип данных может предложить.

Диапазон и точность типов данных с плавающей точкой согласно **стандарту IEEE 754**:

Размер	Диапазон	Точность
4 байта	от ±1.18 x 10^{-38} до ±3.4 x 10^{38}	6-9 значащих цифр (в основном 7)
8 байт	от ±2.23 x 10^{-308} до ±1.80 x 10^{308}	15-18 значащих цифр (в основном 16)
12 байт	от ±3.36 x 10^{-4932} до ±1.18 x 10^{4932}	18-21 значащих цифр
16 байт	от ±3.36 x 10^{-4932} до ±1.18 x 10^{4932}	33-36 значащих цифр

Может показаться немного странным, что 12-байтовая переменная типа с плавающей точкой имеет тот же диапазон, что и 16-байтовая переменная. Это потому, что они имеют одинаковое количество бит, выделенных для экспоненты (только в 16-байтовой переменной точность будет выше).

Правило: Используйте по умолчанию тип double вместо типа float, так как его точность выше.

Ошибки округления

Рассмотрим дробь 1/10. В десятичной системе счисления эту дробь можно представить, как 0.1. В двоичной системе счисления эта дробь представлена в виде бесконечной последовательности — 0.0001100110011001... Именно из-за подобных разногласий в представлении чисел в разных системах счисления, у нас могут возникнуть проблемы с точностью. Например:

```
#include <iomanip>

int main()
{
    double d(0.1);
    std::cout << d << std::endl; // выводимое значение имеет по умолчанию (6 цифр)
    std::cout << std::setprecision(17);
    std::cout << d << std::endl;
    return 0;
}
```

Результат выполнения программы:

```
0.1
0.100000000000001
```

Первый ход выводит 0.1 (что и ожидаемо). После того, как мы изменили для объекта cout точность вывода до 17 цифр, мы увидели, что значением переменной `d` является не совсем 0.1! Подобное происходит из-за ограниченной в количестве выделенной памяти для переменных типа double, а также из-за необходимости округлятиь числа. Ту фигу мы получили типичную **ошибку округления**.

Подробные ошибки могут иметь неожиданные последствия:

```
#include <iomanip>
#include <iostream> // для std::setprecision()

int main()
{
    std::cout << std::setprecision(17);

    double d2(2.0);
    std::cout << d2 << std::endl;

    double d3(0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1); // должно получиться 2.0
    std::cout << d3 << std::endl;
    return 0;
}
```

Результат выполнения программы:

```
2
0.999999999999999
```

Хотя мы складали, что `d2` и `d3` окажутся равными, но это не так. А что, если бы нам довелось сравнивать эти переменные и, исходя из результата, выполнить определенный сценарий? В таком случае ошибок как не миновать.

Математические операции (например, сложение или умножение), как правило, только увеличивают масштаб этих ошибок. Даже если 0.1 имеет погрешность в 17-й значащей цифре, то при выполнении операции сложения десять раз, ошибка округления перемещается к 16-й значащей цифре.

`nan` и `inf`

Есть две специальные категории чисел типа с плавающей запятой:

- `inf` (или **«бесконечность»**, от англ. **«infinity»**), которая может быть либо положительной, либо отрицательной.
- `nan` (или **«не число»**, от англ. **«not a number»**). ИЗ есть несколько видов (обсуждать все виды здесь мы не будем).

Рассмотрим примеры на практике:

```
#include <iomanip>

int main()
{
    double zero = 0.0;
    double posinf = 5.0 / zero; // положительная бесконечность
    std::cout << posinf << "\n";

    double neginf = -5.0 / zero; // отрицательная бесконечность
    std::cout << neginf << "\n";

    double nan = zero / zero; // не число (неопределенное неопределено)
    std::cout << nan << "\n";

    return 0;
}
```

Результат выполнения программы:

```
inf
-inf
-nan(ind)
```

`inf` означает «бесконечность», а `ind` означает «неопределенный» (от англ. **«indeterminate»**). Обратите внимание, результаты вывода `inf` и `nan` зависят от компилятора/ергатуры компьютера, поэтому вывод результатов выполненной программы может отличаться от моего результата.

Заключение

Переменные типа с плавающей точкой отлично подходят для хранения очень больших или очень маленьких (в том числе и дробных) чисел до тех пор, пока они имеют ограниченное количество точных цифр (не требуется точность определенного типа данных).

Переменные типа с плавающей точкой могут иметь небольшие ошибки округления, даже если точность типа не превышена. В большинстве случаев такие ошибки остаются незамеченными, так как они сол стоит значительно. Но следует помнить, что сравнение переменных типов с плавающей точкой может иметь непредвиденные последствия(результаты (а выполнение математических операций с такими переменными может только увеличить масштаб этих ошибок).

Тест

Запишите следующие числа в экспоненциальной записи в стиле языка C++ (используя букву `e`, как символ экспоненты) и определите, сколько значащих цифр имеет каждое из следующих чисел:

- 34.56
- 0.004400
- 123.005
- 140000
- 140000.001
- 0.00000000
- 34500.0

Ответ

- 3.456e1 (4 значащие цифры).
- 4.400e-3 (4 значащие цифры).
- 1.23005e2 (6 значащие цифр).
- 1.4e5 (2 значащие цифры).
- 1.400000e5 (9 значащие цифр).
- 8e-10 (1 значащая цифра). Здесь мантисса не `0.8`, а `8`, поэтому число и имеет только 1 значащую цифру.
- 3.45000e4 (5 значащих цифр). Здесь конечные нули не игнорируются, так как в исходном числе есть точка, которая разделяет целую и дробную части. Хотя это точка никак не влияет на сам число, она влияет на его точность. Если бы исходное число было указано, как `34500`, то ответ равнялся бы `3.45e4`.